

```
=====
=====
PROPUESTA: MÓDULO INVENTORY
Backend REST API – Gestión de Almacenes
Fecha: 2026-05-20
=====
=====
```

CONTEXTO DEL DOMINIO

El módulo gestiona el catálogo de productos del almacén, sus categorías y los movimientos de stock (entradas y salidas de inventario).

Tres entidades principales:

- Category – categorías que agrupan los productos
- Product – artículos almacenados con precio, SKU y nivel de stock
- StockMovement – registro histórico de cada movimiento de inventario

```
=====
=====
PATRONES HEREDADOS DEL MÓDULO AUTH
=====
=====
```

Capa	Convenciones aplicadas
Model	@Data @Builder @NoArgsConstructor @AllArgsConstructor @Builder.Default para valores por defecto @Column con nullable / unique / length explícitos
Repository	@Repository + JpaRepository<Entidad, Long> Query methods derivados (findBy, existsBy, findByActiveTrue)
DTO Request	Lombok completo + validaciones Jakarta (@NotBlank, @NotNull, @Size, @Min, @DecimalMin)
DTO Response	Lombok completo, sin validaciones, nunca expone campos sensibles
Mapper	@Mapper(componentModel = "spring") @Mapping(ignore = true) para id, createdAt, active y campos resueltos por el servicio

Service Interfaz + Impl separados
 @Service @Transactional @RequiredArgsConstructor
 @Transactional(readOnly = true) en métodos de solo
 lectura
 negocio RuntimeException con mensajes claros ante errores de

Controller @RestController @RequestMapping("/api/v1/...")
 @RequiredArgsConstructor
 @Valid @RequestBody en todos los endpoints de entrada
 ResponseEntity fluido: 201 Created / 200 OK / 204 No

Content

```
=====
=====
1. MODELS - modules/inventory/model/
=====
=====
```

MovementType.java (enum)

IN → entrada de mercancía al almacén
 OUT → salida de mercancía del almacén

Category.java

@Entity @Table(name = "categories")

<u>Campo</u>	<u>Tipo</u>	Restricciones
id	Long	PK, auto-incremental
name	String	NOT NULL, UNIQUE, max 80 chars
description	String	max 255 chars, nullable
active	boolean	NOT NULL, default = true

Product.java

@Entity @Table(name = "products")

<u>Campo</u>	<u>Tipo</u>	Restricciones / Notas
id	Long	PK, auto-incremental
sku	String	NOT NULL, UNIQUE, max 50 chars
name	String	NOT NULL, max 120 chars
description	String	max 255 chars, nullable

unitPrice	BigDecimal	NOT NULL, precision 10, scale 2
stockQuantity	Integer	NOT NULL, default = 0
minStockLevel	Integer	NOT NULL, umbral de alerta
active	boolean	NOT NULL, default = true
createdAt	LocalDateTime	NOT NULL, no actualizable,
default = now()		
category	Category	FK → categories.id, LAZY (LAZY evita N+1 queries al listar)

StockMovement.java

```
@Entity @Table(name = "stock_movements")
```

<u>Campo</u>	<u>Tipo</u>	Restricciones / Notas
id	Long	PK, auto-incremental
product	Product	FK → products.id, LAZY
type	MovementType	NOT NULL, almacenado como STRING (IN/OUT)
quantity	Integer	NOT NULL, mínimo 1
reason	String	max 255 chars, nullable
createdAt	LocalDateTime	NOT NULL, no actualizable,
default = now()		

```
=====
=====
2. REPOSITORIES – modules/inventory/repository/
=====
=====
```

CategoryRepository.java

```
extends JpaRepository<Category, Long>

Optional<Category> findByName(String name)
Boolean existsByName(String name)
List<Category> findByActiveTrue()
```

ProductRepository.java

```
extends JpaRepository<Product, Long>

Optional<Product> findBySku(String sku)
Boolean existsBySku(String sku)
List<Product> findByActiveTrue()
```

```

        List<Product>          findByCategoryId(Long categoryId)
        List<Product>
findByStockQuantityLessThanEqualAndActiveTrue(Integer threshold)
        → productos con stock por debajo del umbral
mínimo

```

StockMovementRepository.java

```

    extends JpaRepository<StockMovement, Long>

    List<StockMovement> findByProductId(Long productId)
    List<StockMovement> findByProductIdAndType(Long productId,
MovementType type)

```

```

=====
=====
3. DTOs – modules/inventory/dto/
=====
=====

```

CategoryRequestDTO.java

```

@NotBlank @Size(max = 80) String name
@Size(max = 255) String description

```

CategoryResponseDTO.java

```

Long      id
String    name
String    description
Boolean   active

```

ProductRequestDTO.java

```

@NotBlank @Size(max = 50)      String      sku
@NotBlank @Size(max = 120)     String      name
@Size(max = 255)               String      description
@NotNull @DecimalMin("0.0")    BigDecimal unitPrice
@NotNull @Min(0)               Integer     minStockLevel
@NotNull                       Long        categoryId
(el servicio resuelve la entidad Category desde categoryId)

```

ProductResponseDTO.java

```

Long      id

```

String	sku	
String	name	
String	description	
BigDecimal	unitPrice	
Integer	stockQuantity	
Integer	minStockLevel	
Boolean	active	
LocalDateTime	createdAt	
CategoryResponseDTO	category	← objeto anidado, no solo el ID

StockMovementRequestDTO.java

@NotNull	Long	productId	
@NotNull	MovementType	type	(IN / OUT)
@NotNull @Min(1)	Integer	quantity	
@Size(max=255)	String	reason	

StockMovementResponseDTO.java

Long	id
Long	productId
String	productName
MovementType	type
Integer	quantity
String	reason
LocalDateTime	createdAt

```
=====
=====
4. MAPPERS – modules/inventory/mapper/
=====
=====
```

CategoryMapper.java

```
@Mapper(componentModel = "spring")

CategoryResponseDTO toResponseDTO(Category category)

@Mapping(target = "id", ignore = true)
@Mapping(target = "active", ignore = true)
Category toEntity(CategoryRequestDTO dto)

@Mapping(target = "id", ignore = true)
@Mapping(target = "active", ignore = true)
void updateCategoryFromDTO(CategoryRequestDTO dto, @MappingTarget
```

Category category)

ProductMapper.java

```
@Mapper(componentModel = "spring", uses = {CategoryMapper.class})
(uses delega Category → CategoryResponseDTO al CategoryMapper)

ProductResponseDTO toResponseDTO(Product product)

    @Mapping(target = "id",          ignore = true)
    @Mapping(target = "stockQuantity", ignore = true) ← gestionado
por el servicio
    @Mapping(target = "active",      ignore = true)
    @Mapping(target = "createdAt",   ignore = true)
    @Mapping(target = "category",    ignore = true) ← resuelto por
el servicio
    Product toEntity(ProductRequestDTO dto)

(mismo conjunto de @Mapping para updateProductFromDTO)
```

StockMovementMapper.java

```
@Mapper(componentModel = "spring")

    @Mapping(target = "productId", source = "product.id")
    @Mapping(target = "productName", source = "product.name")
    StockMovementResponseDTO toResponseDTO(StockMovement movement)

    @Mapping(target = "id",          ignore = true)
    @Mapping(target = "product",     ignore = true) ← resuelto por el
servicio
    @Mapping(target = "createdAt", ignore = true)
    StockMovement toEntity(StockMovementRequestDTO dto)
```

```
=====
=====
5. SERVICES – modules/inventory/service/
=====
=====
```

CategoryService.java (interfaz)

```
CategoryResponseDTO createCategory(CategoryRequestDTO
request)
List<CategoryResponseDTO> findAllActive()
CategoryResponseDTO findById(Long id)
CategoryResponseDTO updateCategory(Long id,
```

```

CategoryRequestDTO request)
    void deleteCategory(Long id) ← soft
delete

```

CategoryServiceImpl.java

```

@Service @Transactional @RequiredArgsConstructor
Dependencias: CategoryRepository, CategoryMapper

createCategory → valida nombre único (existsByName), persiste
findAllActive → @Transactional(readOnly = true)
findById → @Transactional(readOnly = true),
RuntimeException si no existe
updateCategory → busca entidad, aplica updateFromDTO, persiste
deleteCategory → busca entidad, setea active = false (nunca
borra físicamente)

```

ProductService.java (interfaz)

```

ProductResponseDTO createProduct(ProductRequestDTO
request)
List<ProductResponseDTO> findAllActive()
ProductResponseDTO findById(Long id)
ProductResponseDTO findBySku(String sku)
ProductResponseDTO updateProduct(Long id,
ProductRequestDTO request)
void deleteProduct(Long id)
ProductResponseDTO
applyStockMovement(StockMovementRequestDTO request)
List<StockMovementResponseDTO> getMovementsByProduct(Long
productId)
List<ProductResponseDTO> findLowStock()

```

ProductServiceImpl.java

```

@Service @Transactional @RequiredArgsConstructor
Dependencias: ProductRepository, CategoryRepository,
StockMovementRepository, ProductMapper,
StockMovementMapper

createProduct → valida SKU único, resuelve Category por
categoryId,
construye Product con stockQuantity = 0,
persiste
applyStockMovement → busca producto, calcula nuevo stock:
    IN → stockQuantity + quantity
    OUT → valida que no quede negativo, resta

```

quantity

actualiza stockQuantity, registra

StockMovement

findLowStock

→ @Transactional(readOnly = true)

findByStockQuantityLessThanEqualAndActiveTrue

deleteProduct

→ soft delete: active = false

```
=====
=====
6. CONTROLLERS - modules/inventory/controller/
=====
=====
```

CategoryController.java

@RestController @RequestMapping("/api/v1/inventory/categories")

Método	Ruta	Operación	Respuesta
POST	/	createCategory	201 Created +
CategoryResponseDT0			
GET	/	findAllActive	200 OK +
List<CategoryResponseDT0>			
GET	/ {id}	findById	200 OK +
CategoryResponseDT0			
PUT	/ {id}	updateCategory	200 OK +
CategoryResponseDT0			
DELETE	/ {id}	deleteCategory	204 No Content

ProductController.java

@RestController @RequestMapping("/api/v1/inventory/products")

Método	Ruta	Operación	Respuesta
POST	/	createProduct	201 Created +
ProductResponseDT0			
GET	/	findAllActive	200 OK +
List<ProductResponseDT0>			
GET	/ {id}	findById	200 OK +
ProductResponseDT0			
GET	/sku/ {sku}	findBySku	200 OK +
ProductResponseDT0			
PUT	/ {id}	updateProduct	200 OK +
ProductResponseDT0			
DELETE	/ {id}	deleteProduct	204 No Content

POST	/stock	applyStockMovement	200 OK	+
ProductResponseDTO				
GET	/{"id"}/movements	getMovementsByProduct	200 OK	+
List<StockMovementResponseDTO>				
GET	/low-stock	findLowStock	200 OK	+
List<ProductResponseDTO>				

```
=====
=====
RESUMEN DE ARCHIVOS
=====
=====
```

Capa Total	Archivos
Models 4	Category, Product, StockMovement, MovementType (enum)
Repositories 3	CategoryRepository, ProductRepository, StockMovementRepository
DTOs 6	CategoryRequest/Response, ProductRequest/Response, StockMovementRequest/Response
Mappers 3	CategoryMapper, ProductMapper, StockMovementMapper
Services 4	CategoryService/Impl, ProductService/Impl
Controllers 2	CategoryController, ProductController
TOTAL 22	

```
=====
=====
```